



Google Summer of Code 2023

[Cloud Native Buildpacks (CNCF)]

Project: Enhancements for Dockerfiles

Personal Info

Name: Darshan Kumar

Email: itsdarshankumar@gmail.com

Telephone: +91 9872389796

Time Zone: India (UTC+5.30)

GitHub: [itsdarshankumar](https://github.com/itsdarshankumar)

Slack username: Darshan Kumar

University Info

University: Indian Institute of Technology (IIT), Roorkee

Major: Computer Science and Engineering

Current Year: 2nd year (May 2025 expected graduation)

Degree: Bachelor of Technology (B.Tech)

About Me

I am a sophomore at the Indian Institute of Technology, Roorkee, from the Computer Science and Engineering department. I've been involved in software development for the past 1.5 years, with several projects deployed to production. I am a developer at our university's software development group ([SDSLabs](#)), a student-run technical group. I have been involved in software development in various spheres, including cloud and infra development. As a part of the group, we develop software to be used by the general public and organize lectures, workshops, CTFs, and hackathons.

I'm passionate about programming and software development since school and have worked on several personal and group projects since coming to college. In my freshman year, I was introduced to Docker, Kubernetes, and programming in Go. I am enthusiastic about computer science and software development. With good programming, communication, and team coordination abilities, together with a strong background in software development, I am confident that I will be an asset to the community.

Statement of Motivation

What is your motivation for participating in Google Summer of Code?

I have always been excited by the prospect of converting ideas into products with real-world impact and have been captivated by the concept of open source ever since I was first introduced to it. Since then, I have wanted to be a part of this community and have been trying to contribute in the small ways I could by having all my projects on GitHub to welcome contributions to my projects. GSoC provides the perfect opportunity to jump right into the Open Source World with the opportunity to work on awesome and even bigger open-source projects with experienced developers. That's why I am motivated to participate in GSoC, as it will enhance my knowledge and development skills and allow me to work on industry-scale projects under experienced mentors.

Why did you choose Cloud Native Buildpacks, and why this project idea?

I am a member of SDS Labs, a student technical club at IIT Roorkee that constantly tries to innovate and foster software development activities on campus. Two of my seniors in SDS Labs have been core contributors to CNCF Community (Gaurav Genani and Mohit Sharma). They introduced me to Cloud native, containerization technology, and CI/CD pipelines. After reading about these technologies, I was amazed by their possibilities, which excites me. Also, I was amazed to learn about the supportive community of Cloud Native Buildpacks and CNCF by hearing [h3llix](#) (Gaurav Genani) and [Scar26](#) (Mohit Sharma) experience and eventually realizing it myself as I worked with my mentors.

I chose this particular project because I am more comfortable in golang, which perfectly aligns with my interests. Also, I wanted to explore more cloud and containerization technology, and this project provides an excellent opportunity to apply my knowledge and skills on a practical scale. I always wanted to optimize the throughput of software by analyzing its performance bottlenecks. Extending the CLI tool in such a large-scale project helps me take a significant step towards it. This project lies in my sphere of interest; therefore, I am naturally inclined toward it.

What are your expectations from us during and after the successful completion of the program?

I look forward to bonding with the community and contributing valuable contributions to the project. I would be grateful to join the family of Cloud Native Buildpacks and CNCF. I am always open to essential discussions over ideas and their implementations. I would love to receive feedback and suggestions on my contributions, as it will help improve my code quality and help me become a better developer in the long run.

As I mentioned above, your community's projects drive me to make consistent contributions to the work it is doing. The GSoC project allows me to work on the project as a whole, and after its successful completion, I would be keen to take up more important issues and continue my open-source contribution through Cloud Native Buildpacks and CNCF.

What are you hoping to learn?

I expect to build upon my DevOps skill set by setting up a large project. Cloud Native Buildpacks could help me more with learning containerization technology and core cloud-native concepts. I hope to gain a deeper understanding of the phases an app follows during its building and how ``pack`` magically transforms source code into OCI Image silently. My work on the ``pack`` platform would allow me to understand how the ``pack`` platform and ``lifecycle`` work generally and the configurations the ``pack`` currently supports.

Also, staying in continuous touch with qualified people in the community would be an excellent opportunity for me. I am very excited to work on this project, along with the highly supportive Cloud Native Buildpack community, in the coming summers.

Related Experience

What kind of projects have you worked on in the past? What technologies did you use?

- [Gasper](#) is an intelligent Platform as a Service (PaaS) written in golang used to deploy and manage applications and databases in any cloud topology. It builds and runs applications in docker containers directly from the source code.
- [Nymeria](#) is a unified Authorization and Identity Access Management system written in golang. It uses Ory Kratos, an open-source framework, to implement authorization and Identity management of users.
- [De_FL](#) was a team project as a submission for ETHIndia 2022. It is a revolutionary application where you can train an AI model using federated learning accurately over a Blockchain network that simultaneously ensures transparency, verifiability, and anonymity. We built a user case for self-driving vehicles. We implemented it by letting only the aggregator server access the models created by individuals and the key to decrypt all these local models on the blockchain network. We won the Biconomy: Prize Pool and IPFS/Filecoin - General Storage Track - Pool Prize.
- [MetaVid](#) was a team project as a submission for ETHforAll. It is a blockchain-based video platform that aims to provide a secure and efficient subscription-based streaming experience. Our platform uses a subscription model that gets renewed upon the user's subscription, ensuring that only authorized

users can access it over the blockchain network. We won the Superfluid: Prize Pool for this.

- [CSAW CTF](#) is one of the oldest and biggest CTFs. Designed as an entry-level, jeopardy-style CTF, this competition is for students trying to break into the security field and for advanced students and industry professionals who want to practice their skills. We were 3rd in the India Quals as a part of team SDS Labs.
- [DownUnder CTF](#) is the most significant online Australian-run Capture The Flag (CTF) competition. We were 53rd in the world as a part of team InfosecIIITR.
- Being part of several projects under SDS Labs, I am comfortable with Golang, C++, Javascript, Typescript, PHP, Solidity, and Verilog. I have sound knowledge of Docker and Kubernetes. Apart from development, I also try my hands on CTFs(capture the flag security events).

Contributions to Cloud Native Buildpacks

I have worked on implementing several ``pack extension`` features like ``extension package``, ``extension inspect``, ``extension -help``, and extending ``inspect <image>`` for extensions, along with writing test cases for each of these features.

- <https://github.com/buildpacks/pack/pull/1661>- (merged): Implements the functionality for the `pack extension package` command.
- <https://github.com/buildpacks/pack/pull/1611>- (merged): Extends the output for `pack inspect <image>` to show extensions if they are used while building the image.
- <https://github.com/buildpacks/pack/pull/1616>- (merged): Implements the functionality for the `pack inspect <extension>` command.
- <https://github.com/buildpacks/pack/pull/1603>- (merged): Adds a flag(`--help`) to the `pack extension` command.

Open Source Contributions:

- Gasper: I worked on gasper's service, where I implemented Application metrics APIs, Database Logs endpoint, fixed Application Logs endpoint, and resolved some deprecated packages.

<https://github.com/sdslabs/gasper/pull/242>

<https://github.com/sdslabs/gasper/pull/246>

- Nymeria: I Integrated Ory OAuth2 and OpenID Connect support and implemented verification functionalities. I also worked on writing the middleware to check roles for using Ory Kratos, implemented the functionalities to get identities, added hot-reloading support, and tested all the flows.

<https://github.com/sdslabs/nymeria/pull/24>

<https://github.com/sdslabs/nymeria/commit/f4ca42166264dc9e497d53e033d15303429fd95f>

<https://github.com/sdslabs/nymeria/commit/1759e78c342c01cf9d6aab18d22a859d776f4623>

<https://github.com/sdslabs/nymeria/commit/3634a2ad7a686a66839b9fc310e17c710fbfd836>

<https://github.com/sdslabs/nymeria/commit/16951364e5db4f0bd90c13d2c8476baaa17a0e75>

Project Details

<https://github.com/buildpacks/pack/issues/1623>

Overview

This project aims at improving the current methods and implementation of extending the build image as well as run image in **pack** by using **host daemon (Docker)** instead of calling the **extender** phase of the **lifecycle**, thus improving the performance of **pack** as mentioned in [\[RFC #105\]](#).

Extension of build and run Image

Currently, the base image and run image can be extended by **`pack`** by calling the **extender** phase of the lifecycle and using **kaniko** under the hood to create the extended image from the build.Dockerfile and run.Dockerfile, generated during the **generate** phase (a lifecycle phase that happens after detect) by invoking the extensions' **`./bin/generate`** executable script. This project aims to replace **`kaniko`** with the host system's daemon (**Docker**), thus not using the **`extender`** phase of the lifecycle to improve the performance of **`pack`** as it already has access to the host's daemon through which can apply the dockerfiles directly.

Currently, we can do the following extensions in the build and run image:

1. **Switching the Run Image** [\[RFC #0105 Phase 1\]](#) - When we want to switch the run image with the run image having **`curl`** installed in it, we can use a builder (example - [itsdarshankumar/extensions-builder](#)) having curl extension mentioned in its **`builder.toml`**, and use it to switch the run image during **`extender`** lifecycle phase.

```

=> EXTENDING
Running the extender on OS linux with:
Container Settings:
  Args: /cnb/lifecycle/extender -app /workspace -log-level debug
  System Envs: CNB_EXPERIMENTAL_MODE=warn CNB_PLATFORM_API=0.11
  Image: pack.local/builder/6a7364736f716470686d:latest
  User: root
  Labels: map[author:pack]
Host Settings:
  Binds: pack-cache-library_hello-extensions_latest-605c1c945967.build:/kaniko pack-layers-aehjpnwihq:/layers
  pack-app-odquqzbgbj:/workspace
  Network Mode:
[extender] Extending index.docker.io/itsdarshankumar/extensions-builder@sha256:76a1292593cbe51e399761f4c7640791f5b3bc8854e5e73dc6e5446425450876
[extender] Starting build
[extender] Running build for buildpack samples/hello-extensions@0.0.1
[extender] Looking up buildpack
[extender] Finding plan
[extender] Creating plan directory
[extender] Preparing paths
[extender] Running build command
[extender] -> Hello Extensions Buildpack
[extender] Processing layers
[extender] Updating environment
[extender] Reading output files
[extender] Updating buildpack processes
[extender] Updating process list
[extender] Finished running build for buildpack samples/hello-extensions@0.0.1
[extender] Copying SBOM files
[extender] Creating SBOM files for legacy BOM
[extender] Listing processes
[extender] Finished build
Extender took 5.150658184s to execute

```

Here the Extender phase took my system 5.1506s to execute.

2. **Extending the Build Image** [\[RFC #0105 Phase 2\]](#) - When we want to extend the build image with a build image having `tree` installed in it, we can use a builder (example - [itsdarshankumar/tree-extensions-builder](#)) having tree extension mentioned in its `builder.toml`, and use it to extend the build image during `extender` lifecycle phase.

```

=> EXTENDING
Running the extender on OS linux with:
Container Settings:
  Args: /cnb/lifecycle/extender -app /workspace -log-level debug
  System Envs: CNB_EXPERIMENTAL_MODE=warn CNB_PLATFORM_API=0.11
  Image: pack.local/builder/656d736e786163656e70:latest
  User: root
  Labels: map[author:pack]
Host Settings:
  Binds: pack-cache-library_hello-tree-extensions_latest-flaeb21cb86c.build:/kaniko pack-layers-xjbmnepvgo:/layers
  pack-app-qnnrsdlgem:/workspace
  Network Mode:
[extender] Extending index.docker.io/itsdarshankumar/tree-extensions-builder@sha256:52edf7aac78fc5117bfcf366e24805296200f0e9dc81982b4a7a8815c478e1aa
[extender] Found build Dockerfile for extension 'samples/tree'
[extender] Applying the Dockerfile at /layers/generated/build/samples_tree/Dockerfile ...
[extender] time="2023-03-28T22:27:02Z" level=info msg="Found sha256:52edf7aac78fc5117bfcf366e24805296200f0e9dc81982b4a7a8815c478e1aa in local cache"
[extender] time="2023-03-28T22:27:02Z" level=error msg="Error while retrieving image from cache: base@sha256:52edf7aac78fc5117bfcf366e24805296200f0e9dc81982b4a7a8815c478e1aa getting image from path: read /kaniko/cache/base/sha256:52edf7aac78fc5117bfcf366e24805296200f0e9dc81982b4a7a8815c478e1aa: is a directory"
[extender] time="2023-03-28T22:27:02Z" level=info msg="Built cross stage deps: map[]"
[extender] time="2023-03-28T22:27:02Z" level=info msg="Found sha256:52edf7aac78fc5117bfcf366e24805296200f0e9dc81982b4a7a8815c478e1aa in local cache"
[extender] time="2023-03-28T22:27:02Z" level=error msg="Error while retrieving image from cache: base@sha256:52edf7aac78fc5117bfcf366e24805296200f0e9dc81982b4a7a8815c478e1aa getting image from path: read /kaniko/cache/base/sha256:52edf7aac78fc5117bfcf366e24805296200f0e9dc81982b4a7a8815c478e1aa: is a directory"
[extender] time="2023-03-28T22:27:02Z" level=info msg="Executing 0 build triggers"
[extender] time="2023-03-28T22:27:02Z" level=info msg="Building stage 'base@sha256:52edf7aac78fc5117bfcf366e24805296200f0e9dc81982b4a7a8815c478e1aa' [idx: '0', base-idx: '-1']"
[extender] time="2023-03-28T22:27:02Z" level=info msg="Cmd: USER"
[extender] time="2023-03-28T22:27:02Z" level=info msg="Checking for cached layer oci:/kaniko/cache/layers/cached:292c5007a0f04b760cf9f52b78b5e79e70bd925af60aea1bb076681819ba11c8 ..."
[extender] time="2023-03-28T22:27:02Z" level=info msg="Using caching version of cmd: RUN apk update update && apk add tree"
[extender] time="2023-03-28T22:27:02Z" level=info msg="Skipping unpacking as no commands require it."
[extender] time="2023-03-28T22:27:02Z" level=info msg="USER root"
[extender] time="2023-03-28T22:27:02Z" level=info msg="Cmd: USER"
[extender] time="2023-03-28T22:27:02Z" level=info msg="No files changed in this command, skipping snapshotting."
[extender] time="2023-03-28T22:27:02Z" level=info msg="RUN apk update update && apk add tree"
[extender] time="2023-03-28T22:27:02Z" level=info msg="Found cached layer, extracting to filesystem"
[extender] Starting build
[extender] Running build for buildpack samples/hello-extensions@0.0.1
[extender] Looking up buildpack
[extender] Finding plan
[extender] Creating plan directory
[extender] Preparing paths
[extender] Running build command
[extender] -> Hello Extensions Buildpack
[extender] tree v2.0.4 (c) 1996 - 2022 by Steve Baker, Thomas Moore, Francesc Rocher, Florian Sesser, Kyosuke Tokoro
[extender] Processing layers
[extender] Updating environment
[extender] Reading output files
[extender] Updating buildpack processes
[extender] Updating process list
[extender] Finished running build for buildpack samples/hello-extensions@0.0.1
[extender] Copying SBOM files
[extender] Creating SBOM files for legacy BOM
[extender] Listing processes
[extender] Finished build
Extender took 5.777381143s to execute

```

Here the Extender phase took my system 5.777s to execute.

Issues:

- **Buildpacks** do not run as the **root** user. They cannot make arbitrary changes to the filesystem to enhance security. Therefore, base image authors must anticipate the OS-level dependencies that will be needed at build and run-time in advance; this isn't always possible, so we need a feature to extend our build and run images while also not giving any extra privileges. Thus we used ``kaniko`` as our choice to do this job without any privileges in the ``extender`` phase of the lifecycle.
- The next issue arises in the performance of the ``pack`` by the use of ``kaniko`` to extend the build and run image in the ``extender`` phase of the **lifecycle**. ``kaniko`` also requires a manifest to extend an image. Thus currently, we are forced to publish the image into an OCI registry from which we can pull the manifest, which is not suitable for local builds. Additionally, it's an added computation as we are already pulling the base images earlier to the host's daemon and redoing it inside the ``extender`` phase container to extend it. ``kaniko`` builds images can be slower in some cases, as ``kaniko`` has to download all the necessary files to build the image every time it is run.

Solution:

- ``pack`` already has access to the host's daemon (**Docker**), so it can apply the generated dockerfiles (**build.Dockerfile** and **run.Dockerfile**) directly, saving the extended build or runtime base image in the host's daemon itself. Thus it will not need to use the extender phase of the lifecycle. Further, having this feature will allow us to drop the requirement that the builder is published to a registry in case of extension and **Docker** daemon builds.
- On the other hand, the **docker** daemon can cache the layers of an image and only rebuild the layers that have changed, which can make builds faster, thus significantly reducing the runtime for an image extension. In some cases, we would need two separate ``extend`` operations - one for the build image and the other for the run image. They can happen in parallel by extending the build image and then proceeding to the ``build`` phase. Then after extending the run image and the build phase has been completed, we can proceed to the export phase, thus saving a lot of time.

Implementation Details

Redefining the `Run` Function

1. The run function inside ``lifecycle_execution.go`` contains all the implementation of different Phases of the lifecycle that ``pack`` uses.

```
func (l *LifecycleExecution) Run(ctx context.Context, phaseFactoryCreator PhaseFactoryCreator) error {
    phaseFactory := phaseFactoryCreator(l)
    .....
    .....
    .....
    .....
    .....
    if l.platformAPI.AtLeast("0.10") && l.hasExtensions() {
        if l.os == "windows" {
            return fmt.Errorf("builder has an order for extensions which is not supported for Windows builds")
        }
        // Verify if you are running with publish false
        if !l.opts.Publish {
            l.logger.Info(style.Step("EXTENDING"))
            // Calling new Extend function that uses hosts docker daemon to extend build and run image
            if err := l.Extend(ctx,.....); err != nil {
                return err
            }
        }
    }
    .....
    .....
    .....
    .....
    .....
}
```

2. Firstly, we extend this **Run** function by introducing another check by the conditional statement to ensure that ``--publish=false`` as we first implement this for the local builds.
3. Next, we call the re-defined version of the **Extend** function that takes context and other necessary arguments as required.

Redefining the `Extend` Function

1. Extend function inside `lifecycle_execution.go` contains all the implementations through which we can extend the base build image and run image using the host's docker daemon.

```
func (l *lifecycleExecution) Extend(ctx context.Context,..... ) error {
    // Create a build context for the Docker image
    buildContext, err := archive.ReadDirAsTar(dir,"base_path",uid ,gid ,mode, normalizeModTime, includeRoot,
    FilterFunction)
    if err != nil {
        panic(err)
    }

    // Build the Docker image
    buildOptions := dockertypes.ImageBuildOptions{
        Context:    buildContext,
        Dockerfile: "Dockerfile",
        Tags:       []string{"extend-build-image/extend-run-image"},
    }

    response, err := l.docker.ImageBuild(ctx, buildContext, buildOptions)
    if err != nil {
        panic(err)
    }
    defer response.Body.Close()

    // Show logs
    _, err = io.Copy(os.Stdout, response.Body)
    if err != nil {
        panic(err)
    }
}
```

2. This function will create a build context that refers to the set of files and directories used to build the image using the ReadDirAsTar function from the "github.com/buildpacks/pack/pkg/archive" package.

```
// Build Options
buildOptions := dockertypes.ImageBuildOptions{
    Context:    buildContext,
    Dockerfile: "build.Dockerfile/run.Dockerfile"
    Tags:       []string{"extend-build-image/extend-run-image"},
}
```

3. In the buildOptions, The `Dockerfile` field in this struct specifies the name of the Dockerfile for building the image. We have to set it as `**build.Dockerfile**` for the extension of the build image and `**run.Dockerfile**` for the extension of the run image. The Docker engine will look at this in the build context directory and use it to build the extended image.

Extending the `DockerClient` Interface

1. **DockerClient** Interface inside `[docker.go](https://github.com/docker/docker-go)` defines all the different methods of the Docker client library.

```
type DockerClient interface {
    .....
    .....
    ImageBuild(ctx context.Context, buildContext io.Reader, options types.ImageBuildOptions)
    (types.ImageBuildResponse, error)
}
```

2. A new method is added to this **ImageBuild** for building images.
3. The **ImageBuild()** method is then called in the **Extend** function inside `lifecycle_execution.go`, passing in the context, the tar archive of the build context, and the build options.

Extending it for Multiple Dockerfiles

1. We need to modify the buildOptions as mentioned in `Redefining Extend Function` that will be able to handle Multiple Dockerfiles by adding an ARG `base\_image` in the ARGS field.

```
// Build Options for multiple extensions
buildOptions := dockertypes.ImageBuildOptions{
    Context:    buildContext,
    Dockerfile: "build.Dockerfile/run.Dockerfile"
    Tags:       []string{"extend-build-image/extend-run-image"},
    Args:       map[string]*string{"base_image": &baseImageTag},
}
```

2. The idea is that when multiple Dockerfiles are applied, the intermediate image generated from the application of the current Dockerfile will be provided as the `base_image` ARG to the next Dockerfile.

```
func (l *LifecycleExecution) Extend(ctx context.Context,..... ) error {

    .....

    .....

    .....

    // Extract the image ID from the output of the build
    // This assumes that the image ID is the last line of the output
    imageID := ""
    output := response.Body.(io.ReaderAt)
    buf := make([]byte, 1024)
    for n, err := output.ReadAt(buf, output.Size()-1024); n > 0 && err == nil;
    {
        imageID = string(buf[:n])
        n, err = output.ReadAt(buf, output.Size()-1024+int64(n))
    }

    // Set the baseImageTag to the image ID of the current build
    baseImageTag = imageID

    .....

    .....

    .....
}
```

3. Hereafter, building each Dockerfile, we extract the image ID from the output of the build and set the value of `baseImageTag` to this ID so that it can be used as the `base_image` value for the next Dockerfile.

Discussion is needed on implementation and how to do the extra step of setting ``io.buildpacks.rebasable`` to the appropriate value for run Image:

Proposed way:

When the run image is extended and ``io.buildpacks.rebasable=true``, the extend phase will communicate to the export phase, the top layer (`runImage.topLayer`) of the run image

(prior to extension) so that the exporter can set the appropriate value of ``io.buildpacks.lifecycle.metadata``.

Deliverables

Replace the ``extend`` phase or kaniko with the host's daemon (Docker) for the extension work:

- The host's daemon (**Docker**) will apply the generated dockerfiles (**build.Dockerfile** and **run.Dockerfile**) directly, saving the extended build or runtime base image in the host's daemon itself.
- Drop the requirement that the builder is published to a registry for extension of the images.
- Setting ``io.buildpacks.rebasable`` to the appropriate value after inspecting the intermediate value after each Dockerfile is applied for the run image case.
- Writing the test cases for the implementations.
- Documenting the implementations.

Usage of Extension in ``pack`` by Host's Daemon (Docker)

- Users may wish to use image extensions in a faster way with better caching and layering if they wish to provide the flexibility of modifying base images dynamically.
- Remove the added dependency of ``kaniko`` while already having access to the host's daemon.
- Retaining all other benefits of Extensions as well.

Project Timeline

Pre GSoC Period	
April 5th - May 3rd	• I will be completing my remaining PRs on Cloud Native Buildpacks and will try to complete my assigned issues before the community bonding

	period. Resolving new issues and reading more about the pack's architecture
Community Bonding Period	
May 4th - May 28th	- Get to know more about the Cloud Native Buildpacks community. - Discussion with my mentor to discuss methods of implementation. - Identify more relevant parts in the codebase
Coding Period	
May 29th - June 5th	- Getting familiar with existing implementations of the lifecycle by `pack` - Create models for Extend function
June 6th - June 19th	- Work on the drafting of Extend Function - Add the new Extend function to the existing pipeline
June 20th - June 21st	Buffer time to complete previous tasks(if any)
June 22nd - July 7th	Start working on different approaches for separating `extend` operations for things to happen in a parallel way in case of both build and run image extension
July 8th - July 9th	Buffer time to complete previous tasks(if any)
July 10th - July 14th	Evaluation 1 Phase
July 15th - July 23rd	Work on the implementation of applying multiple Dockerfiles in the extension
July 24th - August 8th	Work on implementation of setting `io.buildpacks.rebasable` to the appropriate value in sequential Multiple Dockerfile



9th August - 18th August	• If needed, CLI UX fixes for the restructured extender. • Writing tests and documentation if required.
19th August - 20th August	Buffer time to complete previous tasks(if any)
21st Aug - 28th Aug	Debug and Testing

Availability

I can dedicate more than 40 hours a week. The college's summer vacations are scheduled from mid-May to July. Therefore, there would be no classes for the first two months of the program's coding period and no exams throughout the coding period. My college will reopen on July 18th. During this time, I could work more than 25 hours a week.

I generally work from 4 PM to 5 AM IST (6:30 AM to 7:30 PM Eastern Time), although I find it flexible to adjust my working hours if required.

Other than this project, I have no commitments/vacations planned for the summer. I shall keep my status posted to all the community members every week and maintain transparency in the project.

Why me?

I've been quite enthusiastic and eager to learn new things and try my hand at new technologies. I've already worked on a couple of cloud projects like Gasper, Nymeria and several hackathon projects and have been involved in programming in Go for over one year; hence, I can apply the same knowledge and experience drawn from there. I am confident that I can contribute at par with the standards an open-source organization demands.

Apart from this, I would work with many professionals as my mentors. This will be a huge learning experience for me. Having worked in numerous teams and open source projects, I am aware of the software development process and the best practices and recommendations.

Involvements after GSoC

I have already learned a lot contributing to the Cloud Native Buildpacks. Even after the GSoC period ends, I plan on contributing to this organization by adding to my past projects and working on open issues because of my familiarity with the technical stack and the new challenges that I am continually offered in the process.

Also, I would like to complete the secondary goals mentioned in my proposal. Having picked up many development skills, my primary focus would be to help the project and the community grow and also to explore other platforms of Cloud Native Buildpacks like kpack.

Ideas to work on after my primary goals are completed:

These are some of my stretch goals.

1. Extending this functionality to the ``creator`` phase of the **lifecycle** for the ``pack.``

Pros: In this case, all the sequential phases can be executed in a single command.

2. Extending this for remote builds (i.e., ``--publish=true``) by generating the manifest for the extended run image.

Pros: Finals builds can then be pushed to a registry.

References

- <https://buildpacks.io/docs/concepts/>
- <https://buildpacks.io/docs/features/dockerfiles/>
- <https://github.com/buildpacks/pack/issues/1623>
- <https://github.com/buildpacks/rfcs/issues/224>
- <https://github.com/buildpacks/rfcs/blob/main/text/0105-dockerfiles.md>
- <https://buildpacks.io/docs/extension-author-guide/create-extension/>
- <https://buildpacks.io/docs/reference/spec/migration/platform-api-0.9-0.10/>

- 
- https://cloud-native.slack.com/archives/C0331B61A1Y/p1679524484191059?thread_ts=1678793504.988129&cid=C0331B61A1Y
 - https://cloud-native.slack.com/archives/C0331B61A1Y/p1679581119943699?thread_ts=1678793504.988129&cid=C0331B61A1Y
 - https://cloud-native.slack.com/archives/C0331B61A1Y/p1679602246690119?thread_ts=1678793504.988129&cid=C0331B61A1Y